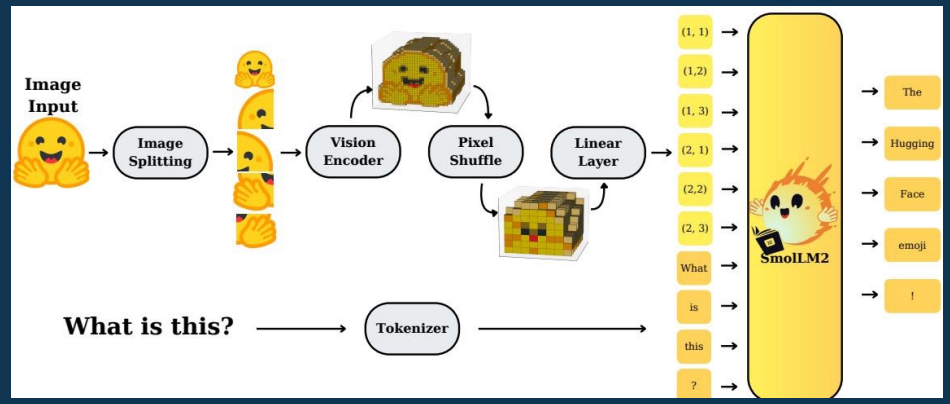
SmolVLM

Redefining Small and Efficient Multimodal Models

Compact VLMs for mobile and edge inference through balanced compute, token compression, and context scaling.

Hugging Face & Stanford University

256M–2.2B	<1GB	16×
model family	smallest-model RAM	r=4 visual-token cut



Architecture pipeline

Images → SigLIP encoder → pixel shuffle → projection → SmolLM2

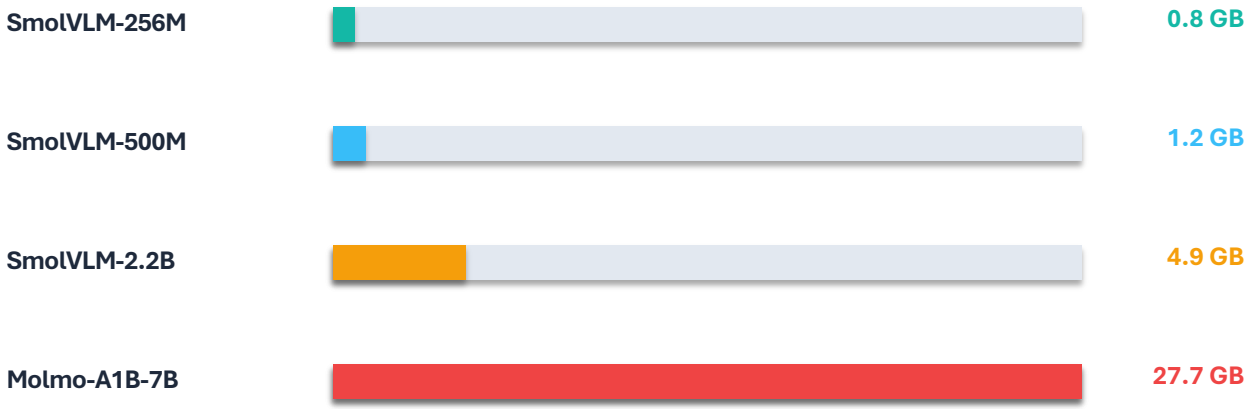
PROBLEM

Large VLMs Are Powerful but Impractical for Edge Deployment

Constraint mismatch

- State-of-the-art VLMs demand large compute and memory footprints.
- Mobile and edge deployments require low RAM, local execution, and predictable latency.
- SmolVLM targets competitive multimodal capability under tight resource budgets.

GPU RAM at batch=1



300×
larger baseline
SmolVLM-256M outperforms Idefics-80B

59.8%
Average score for SmolVLM-2.2B

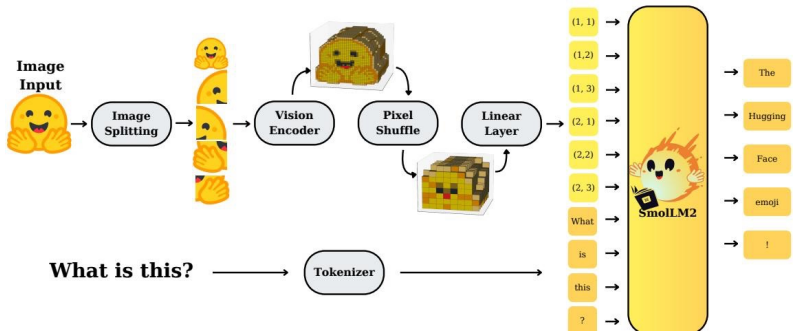
27.7GB
Molmo-A1B-7B memory requirement

0.8GB
SmolVLM-256M RAM

Design question: which architectural choices convert scale into deployable efficiency?

ARCHITECTURE

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1 Split

Input images are split into sub-images; video frames follow the same path.

2 Encode

Sub-images pass through a SigLIP vision encoder.

3 Compress

Pixel shuffle performs space-to-depth token reduction.

4 Fuse

A linear layer projects visual features into SmolLM2 token space.

Output: concatenated visual and text embeddings feed a compact language backbone.

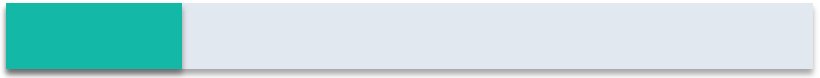
FINDING 1

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Experiment: same 135M language model, different SigLIP encoders

SigLIP-B/16

93M encoder params



Outperforms with 135M LM

SigLIP-SO400M

428M encoder params

Larger encoder underperforms at smallest LM scale

Design implication

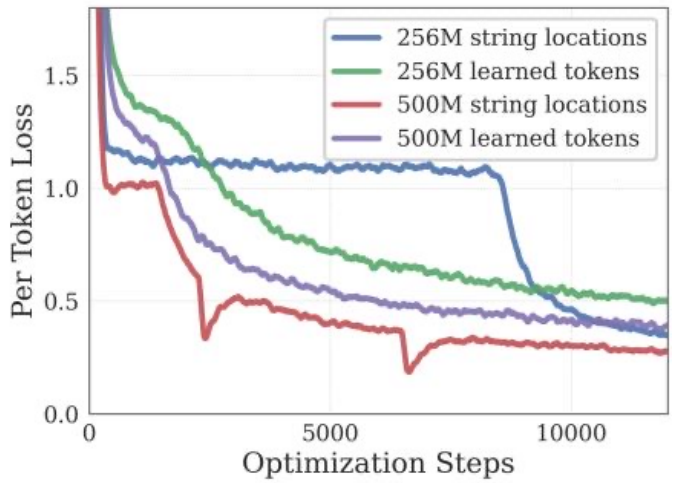
For compact VLMs, allocate parameters to the LM backbone and attention budget before scaling the vision tower.

Scale caveat

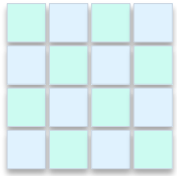
Only at 1.7B LM scale does the larger encoder justify its 10% parameter overhead.

FINDING 3

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models



Space-to-depth compression



16 spatial tokens

1

token
+ channels

$r = 4 \Rightarrow r^2 = 16\times$ fewer visual tokens

Small VLMs

benefit from r=4, reducing attention overhead and improving long-context modeling.

Larger VLMs

use r=2 in the reported design trade-off.

Core effect: fewer visual tokens lower attention cost without abandoning high-resolution inputs.

FINDING 2

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Context scaling strategy

- Increase RoPE base from 10K to 273K.
- Fine-tune on mixed long- and short-context data.
- Use 16K context for 2.2B; 8K context for smaller variants.

Observed effect

Performance improves significantly when context extends from 2K to 16K tokens, enabling more visual tokens from higher-resolution images.

BENCHMARKS

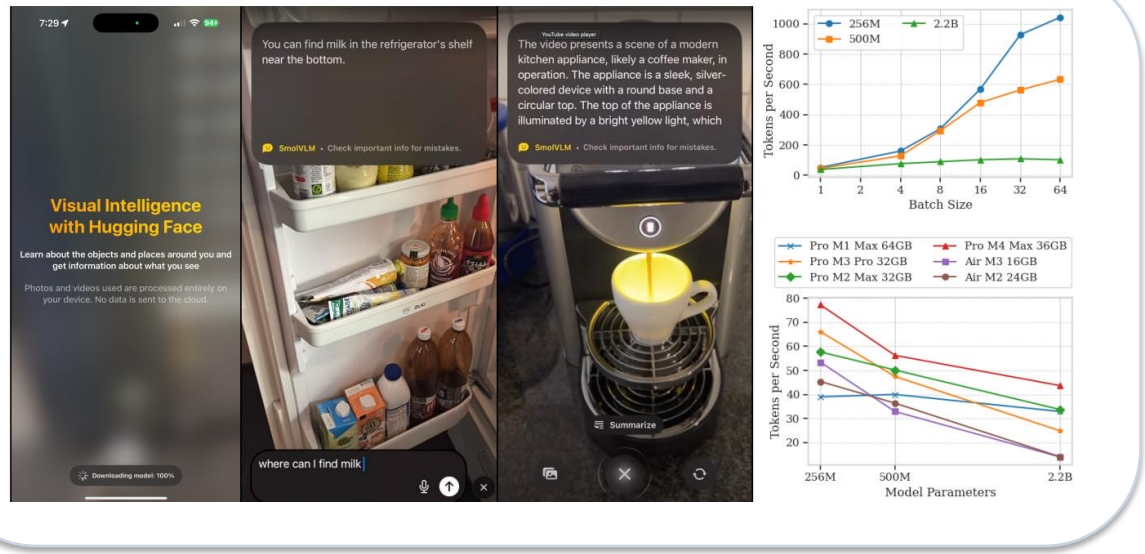
SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% Molmo-A1B-7B
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% Molmo-A1B-7B
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% Molmo-A1B-7B
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% Molmo-A1B-7B
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% Molmo-A1B-7B
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% Molmo-A1B-7B
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% Molmo-A1B-7B
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% Molmo-A1B-7B
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% Molmo-A1B-7B
Video	Video-MME	33.7%	42.2%	52.1%	45.0% InternVL2-2B
Video	MLVU	40.6%	47.3%	55.2%	48.2% InternVL2-2B
Average	**All Benchmarks**	**44.0%**	**51.0%**	**59.8%**	**—**
RAM	**batch=1**	**0.8 GB**	**1.2 GB**	**4.9 GB**	**27.7 GB Molmo-A1B-7B**

SmolVLM-256M uses 0.8GB RAM and surpasses the 300× larger Idefics-80B.

Averages span the benchmarks listed in the source table.

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



HuggingSnap runs photos and videos locally with no cloud dependency.

~80
tokens/sec
SmolVLM-256M on Pro M1 Max 64GB

~55
tokens/sec
256M on MacBook Air M3 16GB

~1000
tokens/sec
256M at batch size 64 on high-end devices

Scaling
Throughput scales roughly linearly with batch size for the smallest model.

- All processing stays on-device.
- The smallest model supports real-time interactive use.
- Batching unlocks high-throughput local workloads on Apple Silicon.

DESIGN PRINCIPLES

Key Takeaways: Design Principles for Efficient Multimodal Models

1

Balance encoder–LM compute

Compact VLMs benefit from smaller vision encoders; scale the vision tower only when the LM is large enough.

2

Compress visual tokens aggressively

Use r=4 pixel shuffle for small VLMs to cut visual tokens by 16× and reduce attention overhead.

3

Extend context deliberately

Move beyond 2K context with RoPE-base adjustment and mixed long/short fine-tuning.

4

Optimize for memory first

Sub-1GB inference is achievable: SmolVLM-256M runs at 0.8GB RAM.

5

Open the full stack

Models, data, code, and HuggingSnap mobile app are fully open-sourced for reproducibility.

Result

Competitive multimodal performance from 256M to 2.2B parameters at 0.8–4.9GB RAM.

Implication: efficient multimodal systems are an architecture problem, not just a scaling problem.